

Kya Bolti Public

A phone in a Mumbai kaali-peeli that lets strangers leave each other messages

Build log: what we made, how we made it, and what changed along the way

Team Kulfi Krabs — Khushi, Devashree, Angad

Mentor: Kushal Bhor

Kulfi Collective — Play It Forward residency

October 2026

1. The idea

Kya Bolti Public is a vintage push-button telephone mounted inside a Mumbai taxi. A passenger who picks up the handset hears a short introduction, then a message left by a stranger who sat in the same seat before them. They can keep listening to more strangers, or press a key and leave a message of their own for whoever comes next. The phone is self-contained: no internet, no screen, no attendant. It lives on a power bank and a micro-SD card.

The project rests on three pillars we set at the start: empathy (hearing a stranger's voice, not a text), connection (the message you leave will be heard by someone you will never meet) and time (the archive grows ride by ride, so the phone slowly becomes a record of the city talking to itself).

2. How the call works

The user flow went through several versions before it settled. The version we are building is below; the changes that led to it are in section 8.

- Lift the handset. The phone asks the passenger to pick a language: 1 English, 2 Hindi, 3 Marathi. If nobody presses anything for ten seconds it asks once more, then defaults to Hindi.
- A short recorded introduction plays: "This phone connects you to the people of Mumbai...". There is no choice here.
- A stranger's message plays, picked at random from the archive. Within one call no message is repeated; once a passenger has heard every message the list resets.
- A single beep marks the end of the message, then the menu: press 4 for another message, press 5 to record your own. Pressing 4 loops back as many times as the passenger likes.
- Pressing 5: "Please record your message after the beep", a beep, thirty seconds of recording, and a beep to say the time is up. The take plays back, then "Press 6 to save, press 7 to redo". 7 deletes the take and records again (up to three times); 6, or ten seconds of silence, saves it.
- After saving: "Your message is saved", "Enjoy the rest of your ride", and an end-of-call beep. The phone then waits silently for the handset to be put down.
- Hanging up at any point stops everything, deletes any unsaved take and returns the phone to idle. A message only enters the archive when 6 was pressed.

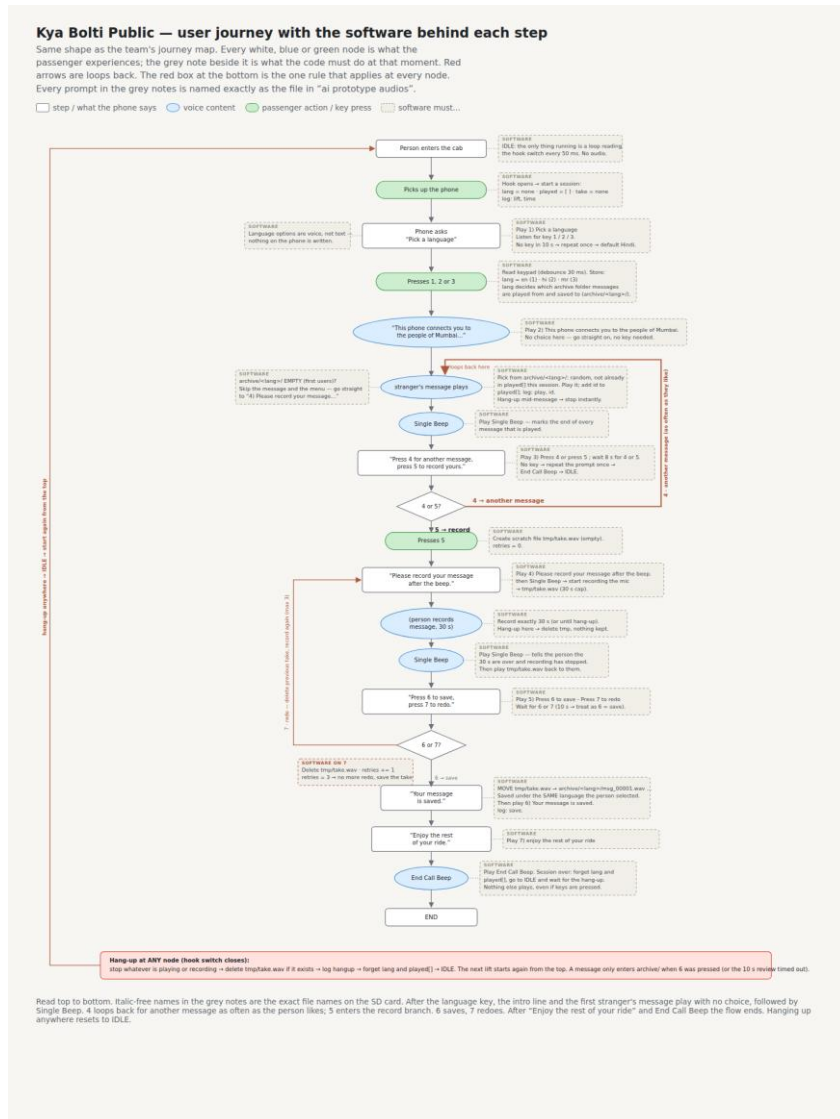


Figure 1 — The final user journey with the software behind each step (flowchart version 6).

3. Hardware

3.1 Why the ESP32 Audio Kit

The first plan was a Raspberry Pi: it would have been the easy software path, but it needs a proper shutdown, draws more power than a power bank likes, and brings a Linux boot time to every power cut. We moved to the Ai-Thinker ESP32 Audio Kit V2.2 (board revision A541), which has everything the phone needs on one board: an ESP32-A1S module, an ES8388 audio codec with a line-in jack and a headphone jack, a micro-SD slot, six push buttons and a 2×7 header with a few spare GPIOs. It boots in a second, runs happily from a 5 V power bank and costs a fraction of the Pi route.

3.2 What is connected to it

Part	What it does	How it is connected
Vintage telephone handset + cradle	The thing the passenger holds	Earpiece on the EARPHONES jack; hook micro-switch on header pin IO22
Telephone keypad (12 keys, 4×3 matrix)	All passenger input	Seven wires: four rows and three columns to GPIOs (see 3.4)

MAX9814 microphone module	Records the passenger	VDD to header 3V3, OUT to the LINEIN jack tip (and ring), GND to board ground
Micro-SD card (FAT32)	Holds the operator recordings and the archive of messages	Onboard slot, 1-bit mode (DIP 1 ON, 2 OFF, 3 ON, 4 OFF, 5 OFF)
EVM 10,000 mAh power bank	Power in the cab	Micro-USB marked POWER; a 47 Ω 1 W resistor across 3V3–GND keeps the bank from switching itself off
Two halves of one 3.5 mm aux cable	Earpiece and microphone leads	Measured with a multimeter: green = tip, white = sleeve, red = ring

The bill of materials was built in the residency’s template (Kulfi_KulfiKrabs_StrangerPhone_BoM.xlsx), every line with a direct product link checked for stock, and came to ₹11,953 against a ₹12,000 budget.

3.3 The hook switch

The cradle switch is a three-terminal micro-switch (COM, NO, NC). COM goes to ground and NO to IO22, so that the handset resting on the cradle holds the pin low and lifting it lets the pin rise. If a cradle happens to work the other way round, NC is used instead, or one setting in the firmware is flipped. Nothing else changes.

3.4 The keypad is a matrix

The telephone keypad turned out not to be twelve independent switches but a 4×3 matrix with seven wires. We found this with a multimeter on continuity: every key connects exactly one of wires 1–4 to one of wires 5–7, which makes 1–4 the four rows of keys and 5–7 the three columns. Key 8, for example, is row 3 + column 6. The seven wires go to the seven GPIOs we have (rows on 13, 19, 23, 18; columns on 5, 21, 36), and the firmware scans them the way the multimeter did — pulling one row low at a time and watching the columns. No resistors or diodes are needed because only one key is ever pressed at a time.

To take the keys off the board we solder a wire to the signal-side leg of each onboard button (the board already has a pull-up resistor on every one) and keep a common ground. A short test sketch (Test K) proves each wire reads HIGH at rest and LOW when the external key is pressed.

Stranger Phone — breadboard prototype (ESP32 Audio Kit V2.2 A541, as photographed)

Red = 3V3 · black = ground · blue = switch signal · green/grey = aux cable wires (green = tip, white = sleeve from your multimeter test). No soldering anywhere.

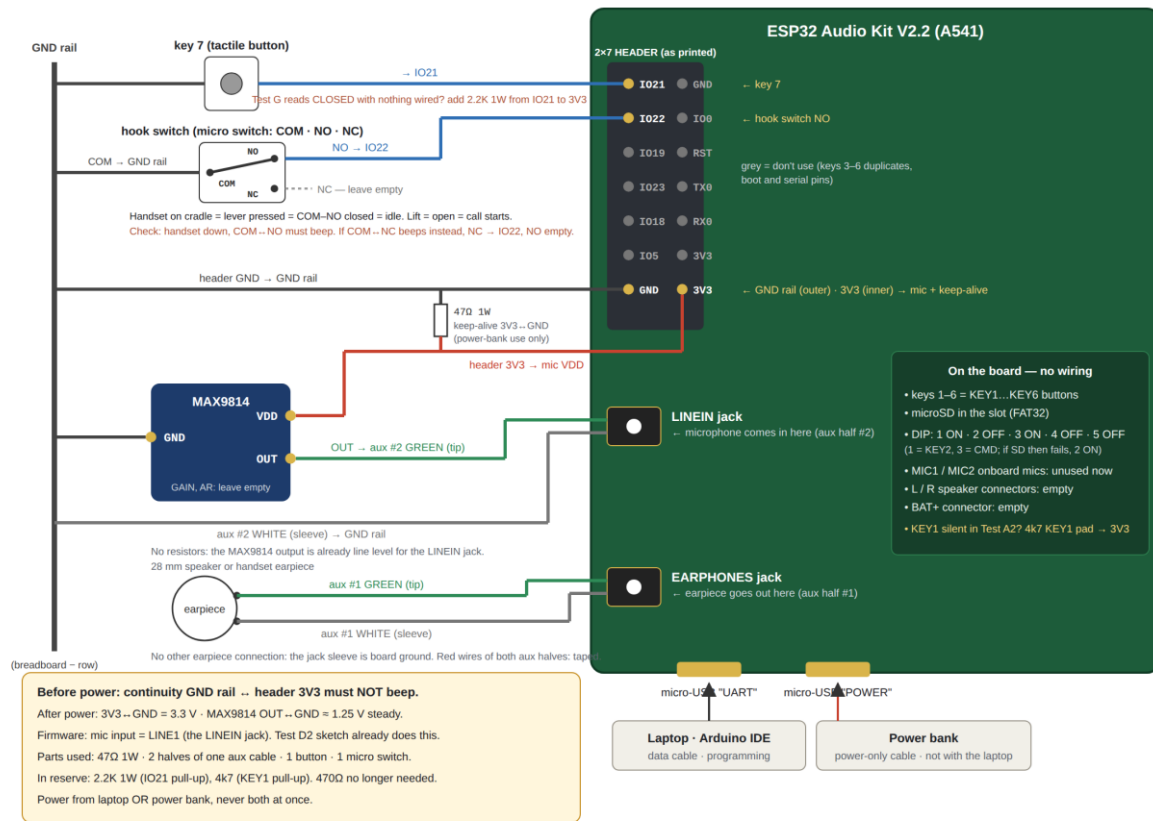


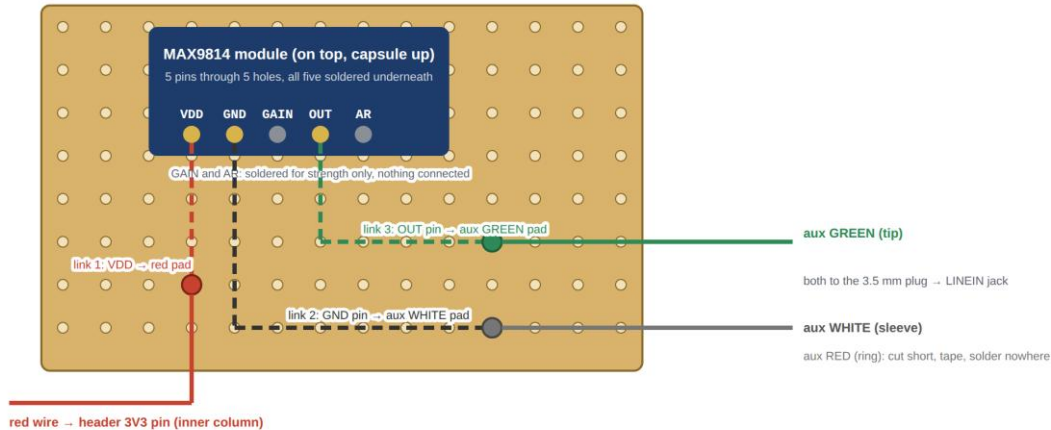
Figure 2 — The breadboard prototype on the A541 board, drawn against the real header and jacks.

3.5 The microphone

The MAX9814 is an electret microphone with its own automatic-gain amplifier, so its output is already at line level and goes straight into the LINEIN jack with no resistors. Early on we planned to solder it to the onboard MIC1 pads through a voltage divider; discovering that the board has a LINEIN jack removed the soldering and the divider in one go. The first working version takes its ground from the earpiece cable's sleeve, which is board ground at the jack; a cleaner version with its own ground wire from the header and the keep-alive resistor on the same board is drawn for later.

MAX9814 on perf board — exactly your breadboard circuit (4 solder joints)

Top view. Every hole is isolated; the dashed lines are the three links you make on the UNDERSIDE. Same wires as the breadboard: red to 3V3, green and white from the aux cable.



Steps

1. Push the module's 5 pins through 5 holes in a row; solder all five on the underside.
2. Pick three free holes a few rows below: one for the red wire, one for the aux GREEN, one for the aux WHITE. Strip 5 mm, tin, push through, solder.
3. On the underside, make the three links with bare wire (or offcut resistor legs): VDD → red pad · GND → white pad · OUT → green pad. Links must not touch each other.
4. Hot-glue the aux cable and the red wire where they leave the board so a tug can't lift a pad.
5. Plug the aux into LINEIN, red wire into header 3V3. That is the whole breadboard circuit, nothing added, nothing removed.

Before powering (continuity): red pad → white pad must NOT beep · green pad → white pad must NOT beep · green pad → red pad must NOT beep.

After powering: red pad → white pad = 3.3 V · green pad → white pad = 1.25 V steady (the MAX9814 is alive).
Ground reaches the mic through the white wire and the jack, exactly as on your breadboard.

Figure 3 — Moving the microphone from the breadboard to a perforated board.

4. Testing one thing at a time

Rather than wire everything and hope, every part got its own small Arduino sketch with every line commented, so a failure pointed at one component. The tests live in the tests folder:

- **A1–A3** the board itself: blink the LED, read the six keys, play a tone through the codec.
- **A4** record from the onboard mics to the SD card (later made configurable, because the driver's naming of the two inputs did not obviously match the board).
- **B** mount the SD card, list it, write and read a file.
- **D / D2** the MAX9814: first a probe test, then recording through the LINEIN jack.
- **E, F** the electret capsule and the speaker.
- **G** the hook switch and the IO21 pin (which is also the board's amplifier-enable line, so the firmware never drives it).
- **I** the keep-alive resistor on the power bank.
- **K** one onboard key routed out to the breadboard.

Along the way we learned the Arduino IDE needs the “ESP32 Dev Module” board with the Huge APP partition, that “Hard resetting via RTS pin” is normal, and that the Serial Monitor must be at 115200 baud.

5. Firmware

The firmware is one Arduino sketch, KyaBoltiPublic.ino, built on the arduino-audio-tools and arduino-audio-driver libraries with arduino-libhelix for MP3. The call flow in section 2 is written almost line for

line as functions: `stepLanguage`, `stepMessage`, `stepRecord` and `session`. Every file it plays is named at the top of the sketch exactly as the recording in the ai prototype audios folder, so changing a prompt means replacing a file, not editing code. Every setting a person might want to change — timeouts, volumes, the record length, which input the microphone is on, which way round the hook switch is — is a commented constant at the top.

Strangers' messages follow one naming rule: any file called User Recording N (MP3 or WAV) counts. Files in the root of the audio folder play for every language; files in the en, hi and mr sub-folders play only to passengers who chose that language, and a new message is saved into the caller's own language folder, numbered on from the highest number already there. Everything the phone does is written to a `log.csv` on the card, and the Serial Monitor prints the same lines plus a listing of the files it can see at boot.

Two small tools sit next to the sketch: `check_sd_files.py`, which checks a local copy of the audio folder for every expected file name and format before the card is copied; and a README describing the card layout.

6. Sound

The first recordings sounded poor. Measuring the files showed why: the operator prompts averaged -25 dB while the test messages peaked at -1 dB, so a digital boost that made the prompts audible was clipping the messages hard. All twelve files were normalised with `ffmpeg` to the same loudness (-16 LUFS, peaks at -1.5 dB), the boost was dropped from $5\times$ to $1.5\times$, and the originals were kept in an `_originals` folder. The firmware applies the boost with a hard limiter so a loud take squashes rather than crackles.

7. Where things stand

- **Working:** power, SD card, playback of all prompts, the language step, recording through the MAX9814, the hook switch, the onboard keys, the volume chain.
- **In progress:** the keypad matrix wiring (continuity map done, soldering next) and the matching scan code; the microphone on perf board; confirming on the Serial Monitor that the three seeded messages are seen on the card.
- **Next:** final handset wiring off the breadboard, operator recordings in all three languages, the enclosure, and a decision on a battery indicator (a USB power bank hides its charge level, so this is either a run-time estimate on an RGB LED or a move to a bare lithium cell with a fuel-gauge chip).

8. Iterations — what changed and why

A short record of the turns, because they explain the shape of what we have.

What	Before	After	Why
Platform	Raspberry Pi	ESP32 Audio Kit V2.2	Instant boot, power-bank friendly, codec + SD + keys on one board
User flow	Language → description → choose listen or record	Language → intro → message → beep → 4/5 menu loop	The intro is not a decision; listening first makes recording feel natural
Flowchart	Swim-lane diagram	Top-down chart mirroring the team's own map, with the 4-loop drawn explicitly	The first version was unclear; the loop was missing

Key 6 / 7 meaning	6 retake, 7 save (code)	6 save, 7 redo	Matched to the prompt as actually recorded
Audio format	16 kHz WAV, per-language prompt folders	MP3 prompts, one set, decoder picked by extension	Recordings came as MP3; one voice for the prototype
Beeps	One beep file	Single Beep after every message and after the 30 s, End Call Beep to close	Passenger needs to know the message ended and the recording stopped
End of call	“Tap for a surprise”, wait 60 s	“Enjoy the rest of your ride” → End Call Beep → silence	Simpler ending; surprise moved to a later layer
Message storage	archive/<lang>/msg_00001.wav	User Recording N in the audio folder, with en/hi/mr sub-folders	Team naming; language-specific playback came back as a need
Microphone	MAX9814 soldered to MIC1 pad via a 4k7/470 Ω divider	MAX9814 into the LINEIN jack, no resistors	The board has a line-in jack; no soldering on the board
Keep-alive resistor	47 Ω ¼ W on 5 V	47 Ω 1 W on 3V3–GND	¼ W would burn; the header has no 5 V pin
Hook switch	Two-terminal switch	COM/NO/NC micro-switch, COM to GND, NO to IO22	The switch we have has three terminals
Keys	Six onboard buttons + one button on IO21	4×3 keypad matrix on seven GPIOs	The vintage keypad is a matrix
Volume	Codec at 80–90 %, then a 5× digital boost	Files normalised, codec at 100 %, 1.5× boost with limiter	Boost was clipping; files were at wildly different levels
Message playback bug	Folder opened with a trailing slash → zero files found	Trailing slash stripped; boot-time file listing added	The phone skipped straight to recording

Each of these is small on its own. Together they are the difference between a plan and a phone that works in the back of a taxi.